

# TNLearn: An Open Source Python Package for Task-based Neurons

Meng Wang<sup>1\*</sup>

WM\_AI@QQ.COM

Tieyun Li<sup>2\*</sup>

TIEYUN.LI@STU.HIT.EDU.CN

Juntong Fan<sup>3\*</sup>

CUHKFANJUNTONG@GMAIL.COM

Hanyu Pei<sup>3</sup>

1155183430@LINK.CUHK.EDU.HK

Jing-Xiao Liao<sup>3</sup>

JINGXIAO.LIAO@CITYU.EDU.HK

Yaodong Yang<sup>4</sup>

YAODONG.YANG@OUTLOOK.COM

Jianwei Ma<sup>2†</sup>

JMA@HIT.EDU.CN

Fenglei Fan<sup>3†</sup>

FENGLFAN@CITYU.EDU.HK

<sup>1</sup>*Shenzhen H&T Intelligent Control Co., Ltd., Shenzhen, China*

<sup>2</sup>*School of Mathematics, Harbin Institute of Technology, Harbin, China*

<sup>3</sup>*Department of Data Science, The City University of Hong Kong, Kowloon, Hong Kong*

<sup>4</sup>*Institute of Artificial Intelligence, Peking University, Beijing, China*

\*Meng Wang, Tieyun Li, and Juntong Fan contribute equally. †Co-corresponding authors

## Abstract

The brain does not rely on a single type of neuron to perform all kinds of tasks; instead, it designs different neurons for different tasks. The concept of task-based neurons represents a paradigm shift compared to task-based architectures. It argues that solving a specific problem requires customized neurons, as task-based neurons capture useful prior knowledge from task-related data. To facilitate the use of task-based neurons in scientific research and industrial applications, we introduce **TNLearn**, an open-source Python package that provides automated construction of task-based neurons and networks, enabling smooth training of task-based networks. Comprehensive documentation, including technical exposition, API reference, and representative examples, is available online. **TNLearn** is open-sourced at <https://github.com/NewT123-WM/tnlearn> and has become a PyTorch ecosystem project.

**Keywords:** Task-based neurons, NeuroAI, symbolic regression, Python package

## 1. Introduction

Over the past few years, many successful networks were due to the novel architecture designs (Srivastava et al., 2015; He et al., 2016; Huang et al., 2017; Fan et al., 2021). These networks almost exclusively use the same type of neurons that are based on the inner-product and the nonlinear activation function. This is in sharp contrast with our brain, which is made up of many functionally and morphologically various neurons (Peng et al., 2021). Moreover, in our brain, all kinds of intelligent behaviors are based on the coordination of different neurons. Therefore, can we introduce different types of neurons into artificial net-

works and examine what merits neuronal diversity can bring (Fan et al., 2025b, 2026)? The core philosophy of introducing neuronal diversity is dimension augmentation”. An artificial network has two highly complementary dimensions: the neuron type and architecture. Designing well-performing neurons represents a paradigm shift relative to designing well-performing architectures. Moreover, the well-performing neurons can be integrated into the well-performing architecture to further boost a model’s representation power.

Along this direction, there is a growing interest in introducing new neurons into deep learning (Chrysos et al., 2021; Fan et al., 2018; Jiang et al., 2020; Mantini and Shah, 2021; Goyal et al., 2020) such as quadratic neurons (Fan et al., 2025a; Jiang et al., 2020; Mantini and Shah, 2021; Goyal et al., 2020; Liao et al., 2023). Xu et al. (2022) developed a package called **QuadraLib** for the optimization of quadratic neurons. Also, Xu et al. (2026) proposed to use quadratic adaptation to train a high-order network. At the same time, high-order and quadratic neurons have been successfully applied in a wide spectrum of applications including medical imaging (Fan et al., 2019), industrial informatics (Tang et al., 2024), bearing fault diagnosis (Liao et al., 2025), clinical imaging diagnosis (Li et al., 2024) and civil engineering (Nguyen et al., 2019). This line of studies largely confirms the feasibility and potential of developing deep learning with new neurons.

Furthermore, we believe that there should be no one-size-fits-all neuron, *i.e.*, the performance of one neuron cannot be the best for all tasks. Biological neuronal diversity emerges from the brain’s necessity to tackle complex tasks. Rather than relying on a single neuron type for all tasks, the brain tailors neurons to specific tasks with neuronal differentiation. Therefore, the concept of task-based neurons differs from that of task-based architectures. The former asserts that customized neurons should be prototyped to address specific problems. Task-based neurons leverage valuable prior knowledge from task-related data, allowing networks composed of such neurons to integrate task-specific priors. Consequently, networks made of task-based neurons should outperform those built from generic neurons with the same structure. While the concepts of task-based neurons and networks have been discussed in previous works (Chrysos et al., 2021, 2022; Fan et al., 2026), limited effort has been devoted to systematically materialize these ideas, methodologies, and algorithms into user-friendly open-source software.

To tackle this challenge, we have developed **TNLearn**, an open-source Python library tailored for creating task-based neurons with different search methods and integrating them into networks. Serving as an initial implementation of task-based neurons, **TNLearn** is crafted to offer a user-friendly solution for beginners while also serving as a cutting-edge benchmark for seasoned researchers. Primarily, **TNLearn** emphasizes user-friendliness, striking a harmonious balance between accessibility and flexibility. The architecture of **TNLearn** is modular, enabling users to selectively incorporate modules that align with their specific needs and workflows. As demonstrated in the code snippet below, building a task-based network in **TNLearn** is straightforward. With only a few lines of code, users can swiftly generate preliminary outcomes. This simplicity not only encourages broad adoption but also enriches users’ machine learning capabilities, bringing this novel tool to neural network novices. Furthermore, **TNLearn** can function as a cutting-edge reference point, allowing users to benchmark their algorithmic designs effectively. We believe that **TNLearn**, as a part of the ecosystem, is instrumental to the research and translation of artificial neural networks.

## 2. Design and Features of TNLearn

Leveraging Python, a popular language in the machine learning community, TNLearn benefits from a rich ecosystem that provides many useful machine learning utilities and supports automatic differentiation. At a high level, TNLearn is designed to offer user-friendly strategies for easy instantiation and training with minimal code. Our implementations undergo rigorous automatic unit testing to ensure reliability. Figure 1 presents a high-level overview of TNLearn’s architecture. Now, we elaborate on each element:

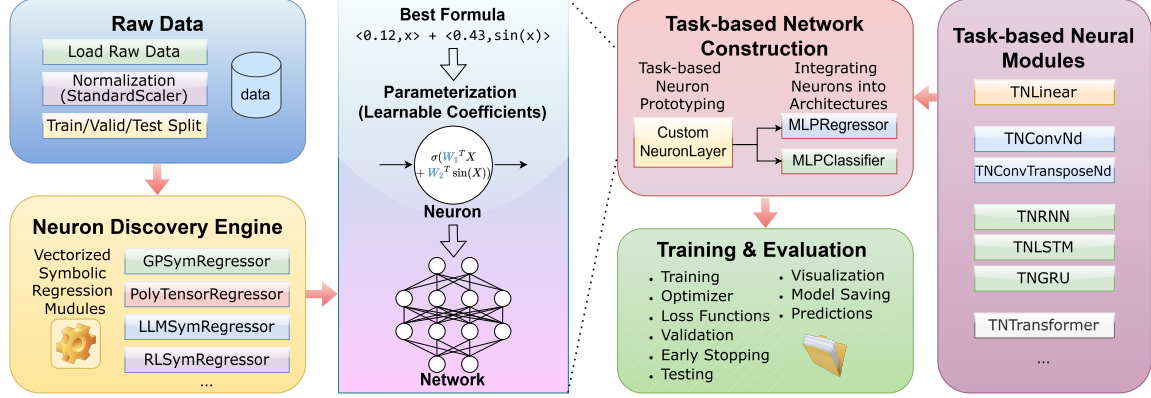


Figure 1: The high-level architecture of TNLearn

### 2.1 Neuron Discovery Engine

**Vectorized Symbolic Regression.** Symbolic regression (SR) is inspired by scientific discoveries in physics (Schmidt and Lipson, 2009; Bartlett et al., 2023). Its goal is to identify optimal formulas that fit the input data by utilizing arbitrary compositions of primitive mathematical operations applied to the input variables, including arithmetic operators and logarithmic, trigonometric, and exponential functions. SR represents a formula as a tree structure and applies genetic programming based on this structure. The vectorized symbolic regression (VSR) forces each input variable to perform the same computation in order to learn a homogeneous formula for downstream parallel computation and avoid the curse of dimensionality. Here we slightly abuse the concept of homogeneity, meaning that all variables undergo identical operations, and the associated coefficients may differ. This module implements VSR from scratch, rather than being a direct import from the existing mature symbolic regression methods. Furthermore, how to find an effective vectorized formula is an optimization problem that cannot be solved by gradient-based methods, since gradients cannot be derived. We have implemented four methods to enrich the armory of our package: GPSymRegressor, LLMsymRegressor, PolyTensorRegressor, and RLSymRegressor. At present, there is no principle guiding which method is superior under which conditions. We encourage users to choose freely among them. In Appendix C, we introduce all these neuron discovery methods in detail for readers’ further reference.

```
from tnllearn import GPSymRegressor
from tnllearn import MLPRegressor
```

```

from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
# Generate data.
X, y = make_regression(n_samples=200, random_state=1)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=1)
# A symbolic regression algorithm is used to generate task-based neurons.
neuron = GPSymRegressor()
neuron.fit(X_train, y_train)

```

`GPSymRegressor` uses the genetic programming to generate random expressions, evaluate their fitness, and evolve expressions through mutation and crossover. It includes all necessary hyperparameters, such as the population size, tournament configuration, number of generations, and more, to assist users in optimizing the learning of formulas. In the example above, we use a `scikit-learn` built-in function `make_regression` to generate synthetic data, and then apply `GPSymRegressor` to learn a homogeneous formula.

`LLMSymRegressor` employs a large language model to discover a formula. During training on massive data, large language models gain the so-called super-intuition for mathematical structures, enabling a direct “guess” at equation forms. In `TNLearn`, this capability is integrated into the vectorized symbolic regression framework via `LLMSymRegressor`. Inspired by `DrSR` (Wang et al., 2025), `LLMSymRegressor` combines data-driven insights with the experience-oriented reflection, generates equation skeletons, and fits coefficients with optimizers like `BFGS`.

`RLSymRegressor` provides an efficient way to discover neuron formulas. It uses a policy network with risk-seeking `REINFORCE` (Williams, 1992) to search over the predefined candidate library, and fits the coefficients of the selected terms via Ridge regression. Users can control the search space via parameters such as the maximum power order  $K$ , the maximum interaction order  $R$ , and the maximum number of sampled terms  $m$  per episode, making it flexible for discovering formulas in various tasks while maintaining computational efficiency. Note that `RLSymRegressor` may be of independent interest to the field of symbolic regression. Note that `RLSymRegressor` may be of independent interest to the field of symbolic regression, as it offers a reinforcement-learning perspective that recasts symbolic optimization as a sequential decision-making problem.

`PolyTensorRegressor` provides a concrete implementation within this framework by restricting the learned formula to a tensor-decomposed representation. It leverages tensor decomposition to efficiently discover symbolic formulas within the vectorized framework by assuming the maximum degree of the learned formula. Specifically, it decomposes the aggregation function into three complementary streams: a polynomial stream for independent feature transformations, a tensor-decomposed interaction stream that captures high-order feature dependencies using Canonical Polyadic (CP) decomposition with low rank (Guo et al., 2011), and a periodic stream for cyclic patterns. By jointly optimizing structural gates and projection matrices through gradient descent, this method provides a concise way to discover effective neurons end-to-end.

## 2.2 Task-based Neuron Construction

This module parameterizes the acquired formula, making the parameters learnable before the formula serves as the aggregation function of the neuron. The learned parameters can

serve as a good initialization, or we can randomly initialize all neurons. Parameterization enables task-based neurons to adapt and interact within a network. Activation functions, such as the ReLU function (Nair and Hinton, 2010), keep intact when connected to a network, as these functions have been widely tested and proven to perform well. This module also encourages users to customize the activation function.

## 2.3 Task-based Network Construction

**Integrating Task-based Neurons into an Architecture.** After the task-based neurons are prototyped, we integrate these neurons into a specified network architecture for regression and classification tasks. This module follows the `scikit-learn` estimator API for data loading, loss definition, and evaluation, and relies on PyTorch for automatic differentiation. An example usage is as follows:

```
# Build neural network using task-based neurons and train it.
clf = MLPRegressor(neurons=neuron.neuron)
clf.fit(X_train, y_train)
# Predict
clf.predict(X_test)
```

**Customizable Neuron Layers for Generic Architectures.** In addition to the multi-layer perceptron, `TNLearn` provides a family of modular layers that cover fully-connected, convolutional, recurrent, and Transformer architectures, with the neuron aggregation function governed by a user-defined symbolic expression. These layers are constructed as immediate substitutes for the corresponding PyTorch modules, permitting their integration into the existing models without altering the surrounding structure. Numeric coefficients in the symbolic expression are ignored, and each layer learns its own weights. This design equips every neuron with a tunable nonlinearity that can reflect prior knowledge of the problem domain or the statistical properties of the data. For instance, a practitioner working on image data can replace a conventional convolution with a customized version by writing the following code:

```
from tnlearn.modules import TNConv2d
conv = TNConv2d(3, 64, 3, symbolic_expression='x + sin(x)')
```

which introduces a sinusoidal inductive bias into the feature extraction process. Similarly, recurrent cells and Transformer encoder layers accept the same symbolic expression, enabling coherent neuron specification across different network components. By exposing the internal operation of the neuron, `TNLearn` facilitates systematic investigation into how alternative aggregation functions affect predictive accuracy and generalization capacity, a direction that remains largely uncharted with fixed activation functions. The library offers thorough documentation and illustrative examples for these advanced layers, thereby supporting rapid prototyping and reproducible research.

`TNLearn` is accompanied by comprehensive documentation that covers technical expositions, task-based neuron guidance, and representative examples. The main resources are summarized in Table 1.

| Resource      | Details                                                                                                 |
|---------------|---------------------------------------------------------------------------------------------------------|
| Installation  | <code>pip install tnlearn</code>                                                                        |
| Documentation | <a href="https://tnlearn-documentation.readthedocs.io">https://tnlearn-documentation.readthedocs.io</a> |
| Source code   | <a href="https://github.com/NewT123-WM/tnlearn">https://github.com/NewT123-WM/tnlearn</a>               |
| PyPI          | <a href="https://pypi.org/project/tnlearn">https://pypi.org/project/tnlearn</a>                         |

Table 1: Quick start resources for TNLearn.

### 3. What Can You Do with TNLearn?

**Model Design.** It can be used to solve real-world problems. Task-based neurons and generic neurons are two different approaches in the field of artificial neural networks. We summarize some potential advantages of task-based neurons over generic neurons, although these advantages of task-based neurons over generic neurons can depend on the specific task and the design of the neural network. i) *Efficiency*: Task-based neurons are designed to perform specific tasks, which can make them more efficient in terms of computational resources and time. ii) *Specialization*: Task-based neurons can be specialized for specific tasks because they incorporate more prior knowledge, which can lead to better performance in those tasks. iii) *Interpretability (mild)*: Task-based neurons can be easier to interpret, as their function is directly related to the task they are designed for.

**Benchmark.** It can be used in neural network research. Task-based neurons can serve as a baseline method for comparison with other models. It serves as a reference point against which the performance of more complex or advanced models can be measured. This is particularly useful in machine learning and data science, where it helps to establish a standard for evaluating the effectiveness of different models.

### 4. Conclusion and Outlook

In summary, we present **TNLearn**, a user-friendly Python package for task-based neurons that aims to be easy to use, versatile for different data, and performant on different tasks. **TNLearn** can engage the interest of a wide audience, since **TNLearn** provides a basic machine learning tool instead of focusing on a specific application field. We plan to keep this package up-to-date by including more architectures such as Mamba (Gu and Dao, 2023), and more utilities. We will also keep this package actively maintained. Recently, EvoGP (Wu et al., 2026) has demonstrated efficient GPU acceleration for tree-based GP through tensorized tree encoding and adaptive parallelism. We also plan to equip GPSymRegressor with GPU parallelization in future releases, enabling it to scale to larger datasets while preserving its lightweight and extensible design for task-based neuron discovery. We enthusiastically welcome contributions and suggestions from the community.

## Appendix A. Project Structure and Installation

### A.1 Project Structure

The TNLearn codebase is organized into a modular directory hierarchy. Core functionality resides in the `tnlearn/` package, which provides the regressor implementations, custom operator logic, network layer construction, and custom module integration. The remaining directories are dedicated to testing, benchmarking, and example usage.

```
tnlearn/
|-- tnlearn/                # Main package
|   |-- __init__.py         # Export public classes: GPSymRegressor,
|                           # MLPRegressor, MLPClassifier,
|                           # LLMSymRegressor, RLSymRegressor, ...
|   |-- base.py             # BaseModel class
|   |-- base1.py            # Legacy base (maintained for compatibility)
|   |-- TN_base.py          # Task-based network base classes
|   |-- operator/           # Customized operators and simplification
|       |-- inner_product.py
|       |-- gp_regressor.py  # Genetic programming for regression
|       |-- poly_regressor.py # Polynomial tensor regression
|       |-- rl_regressor.py  # Reinforcement learning based discovery
|       |-- drsr/           # LLM-based symbolic regression
|           |-- llm.py
|           |-- agent.py
|           |-- evaluator.py
|           |-- ...
|   |-- modules/            # Task-based neural modules
|       |-- TNlinear.py
|       |-- TNconv.py
|       |-- ...
|   |-- mlpregressor.py     # Task-based network for regression
|   |-- mlpclassifier.py    # Task-based network for classification
|   |-- neurons.py          # Neuron expression parsing & parameterization
|   |-- activation_function.py
|   |-- loss_function.py
|   |-- optimizer.py
|   |-- preprocessing.py    # DataPreprocessor
|   |-- utils.py
|-- tests/                  # Unit tests
|   |-- test_regressor.py
|   |-- test_mlpregressor.py
|   |-- test_mlpclassifier.py
|   |-- ...
|-- benchmark/              # Evaluation example scripts
|   |-- regression/
|   |-- classification/
|-- examples/               # Usage examples
|   |-- example_regression.py
|   |-- example_classification.py
|   |-- example_drsr.py
|   |-- example_rl_regression.py
|   |-- data/
```

## A.2 Installation

TNLearn is available on PyPI, and can be installed via `pip`. It requires Python 3.8 or later. The library is built on top of PyTorch, scikit-learn, and NumPy, all of which are automatically resolved during installation.

```
pip install tnlearn
```

For reproducibility, a specific version can be installed as follows:

```
pip install tnlearn==0.1.0
```

Alternatively, the latest development version can be installed directly from the GitHub repository:

```
git clone https://github.com/NewT123-WM/tnlearn.git
cd tnlearn
pip install -e .
```

After installation, the package can be imported as follows:

```
import tnlearn
from tnlearn import GPSymRegressor, LLSymRegressor, RLSymRegressor,
                    PolyTensorRegressor, MLPRegressor,
                    MLPClassifier
```

For GPU support, users should install the PyTorch version compatible with their CUDA version before installing TNLearn.

The documentation, including the API reference and tutorial notebooks, is available at <https://tnlearn-documentation.readthedocs.io>.

## Appendix B. Architecture and Implementation Guide

This appendix walks through the typical workflow of TNLearn, from preparing data to building and training a task-based network. Rather than serving as an exhaustive API reference, it aims to convey the design philosophy behind the library and how its components fit together.

### B.1 General Workflow

The standard pipeline proceeds through a sequence of stages, as illustrated in Figure 1. The workflow is intentionally modular: users may replace any component, including the neuron discovery method, the network architecture, the optimizer, or the loss function, without altering the overall structure.

Together, these stages form an end-to-end, task-oriented pipeline that carries users from neuron discovery to task-specific network construction and training. The process begins with loading and optionally preprocessing the dataset, for example, through the built-in `DataPreprocessor`. If a task-specific neuron is desired, one of the provided symbolic regressors can automatically distill a suitable neuron formula from the training data; alternatively, users may hand-craft a formula and bypass this step. The user then instantiates either `MLPRegressor` or `MLPClassifier` with the discovered or manually specified neuron

expression, together with network hyperparameters such as hidden-layer sizes, activation functions, loss function, and optimizer. For greater architectural freedom, a customized network can also be assembled from `tnlearn.modules`. Once the network is constructed, it is trained by calling `fit` and then evaluated with `predict` and `score` for `MLPRegressor` and `MLPClassifier`. For networks built with `tnlearn.modules`, training follows a PyTorch-style paradigm: users construct the computational graph to define the forward pass and design the loss function, then hand-code the optimization loop.

## B.2 Core Usage Pattern

The following minimal example illustrates the complete pipeline, beginning with the built-in symbolic regressor and then feeding the discovered formula into a task-based MLP regressor. The same structure applies when using `LLMSymRegressor`, `RLSymRegressor`, or `PolyTensorRegressor`, as well as when a manually defined formula is supplied as the neuron input.

```
import numpy as np
from sklearn.model_selection import train_test_split
from tnlearn import GPSymRegressor, MLPRegressor

# 1. Prepare data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 2. Automatically discover a neuron formula
neuron_finder = GPSymRegressor(pop_size=2000, max_generations=15)
neuron_finder.fit(X_train, y_train)
formula = neuron_finder.neuron # e.g., "x**3 + x + 1.0"

# 3. Build task-based network
model = MLPRegressor(
    neurons=formula,
    layers_list=[64, 32],
    activation_funcs='relu',
    max_iter=200,
    batch_size=64,
    lr=0.001
)

# 4. Train and evaluate
model.fit(X_train, y_train)
r2 = model.score(X_test, y_test)
print(f"Test R^2 = {r2:.4f}")
```

## B.3 Integration with Scikit-learn Workflows

TNLearn adheres to the `scikit-learn` estimator API, making it straightforward to integrate into the existing pipelines, grid searches, and cross-validation workflows. For example, a task-based classifier can be passed to `GridSearchCV` just as any standard `scikit-learn` estimator, enabling hyperparameter tuning with minimal additional code.

## B.4 Extending TNLearn

TNLearn is designed to be extended. Users may customize neuron formulas by supplying any valid mathematical expression as the `neurons` argument, following the syntax documented online. For deeper modifications, `CustomNeuronLayer` can be subclassed to define a novel aggregation function, and new activation or loss functions can be registered by extending the dictionaries in `activation_function.py` and `loss_function.py`. The developer documentation provides step-by-step guidance for these customization paths.

## Appendix C. Vectorized Symbolic Regressors in TNLearn

Task-based neuron design aims to discover a symbolic expression that serves as the aggregation function of a neuron. Given an input matrix  $\mathbf{X} \in \mathbb{R}^{d \times N}$  ( $d$  features,  $N$  samples, with each column corresponding to one sample) and a target matrix  $\mathbf{Y} \in \mathbb{R}^{d_{\text{out}} \times N}$  (with columns aligned to the samples), the goal is to find a symbolic expression  $\phi$  (or a combination of expressions) that minimizes the empirical risk:

$$\min_{\phi} \|\mathbf{Y} - \phi(\mathbf{X})\|_F^2, \quad (1)$$

where  $\phi(\mathbf{X})$  is learned by symbolic regression. This reflects the core idea of task-based neuron design: a neuron that already fits the raw data well provides a task-aligned inductive bias, so a network composed of such neurons is expected to fit better than one built from generic neurons. During learning, the symbolic expression may apply a homogeneous univariate operation element-wise to each feature, possibly followed by an aggregation such as summation, or it may include products of inner products to model relationships between features and produce cross-terms.

To automate this discovery, TNLearn provides four complementary regressors: a genetic programming engine (`GPSymRegressor`), an LLM-driven searcher (`LLMSymRegressor`), a reinforcement learning agent (`RLSymRegressor`), and a differentiable tensor-based regressor (`PolyTensorRegressor`). They rely on different search strategies, and can produce homogeneous symbolic formulas, thereby enabling the construction of task-specific neurons with built-in feature interactions.

### C.1 GPSymRegressor

`GPSymRegressor` is a genetic programming engine (Cramer, 1985) that discovers a symbolic expression  $\phi$  in the form of trees. Each candidate solution is a parse tree. Its internal nodes are arithmetic operators  $\{+, -, \times\}$ , unary negation, and the binary inner-product operator  $\langle \cdot, \cdot \rangle$ , while its leaf nodes are either the variable  $\mathbf{x}$  or random real constants from a predefined range. Such a tree can be converted to a string and simplified using `SymPy`. The simplified string is then evaluated by substituting  $\mathbf{x}$  with every sample of the entire input matrix  $\mathbf{X}$ , thereby enabling a vectorized computation.

Since `SymPy` cannot handle expressions containing inner products, we customize the operation logic and simplification rules. The inner-product operator  $\langle \cdot, \cdot \rangle$  takes two sub-expressions, each being a matrix of shape  $d \times N$  or a scalar. If both are matrices, the inner product is computed for each sample as the element-wise sum of products across the feature dimension, yielding a vector of length  $N$ . Scalars, in contrast, are handled by broadcasting.

This operator enables the generation of interaction terms, such as the aggregation function  $(\mathbf{w}_1^T \mathbf{x})(\mathbf{w}_2^T \mathbf{x})$  for each sample vector.

Natural selection is the survival of the fittest. Inspired by biology, we draw lessons from the evolution of species. Since we aim for the formula that best fits the original data, we need to iterate through the genetic algorithm to gradually obtain the tree structure corresponding to the best-fitting formula. Therefore, we define the mean squared error (MSE) between the target and the output as the main component of fitness, and allow genes with lower fitness to be inherited. In addition, to avoid trivial solutions, we discard expressions with fewer than two occurrences of  $\mathbf{x}$ . In genetic operations, parent selection can be based on ranking or tournament selection, both selecting the one with the lowest fitness from a specific sequence.

Each generation has multiple genetic operations. Crossover exchanges random subtrees between two parents with probability  $p_{\text{crossover}}$ , mutation replaces a random subtree with a new one with probability  $0.9 - p_{\text{crossover}}$ , and reproduction copies a parent unchanged with probability 0.1. In addition, the program also sets a random immigration operation, which generates a completely different random tree. Subtree selection is biased toward deep nodes and away from the root to preserve large structures and enable local refinement. To maintain population quality, a small number of elite individuals are retained when entering the next generation. The overall workflow is illustrated in Figure 2.

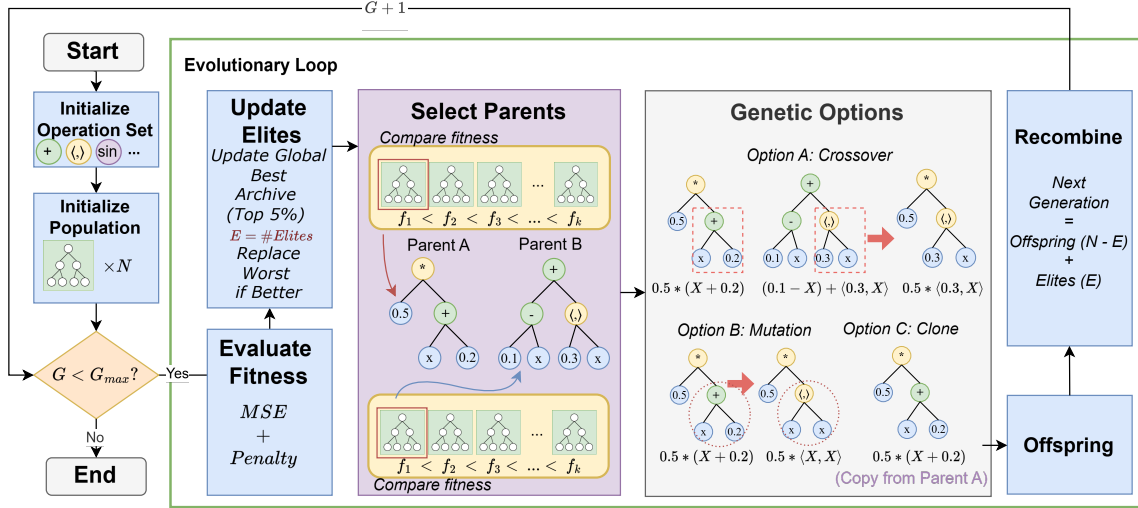


Figure 2: Genetic programming loop of GPSymRegressor, including evaluation, elite preservation, parent selection, genetic operations, and recombination.

After the final generation, the expression  $\phi^*$  with the best (lowest) fitness is returned. The discovered formula provides an interpretable closed-form representation of the learned nonlinear transformation.

## C.2 LLMSymRegressor

LLMSymRegressor leverages a large language model to propose candidate symbolic expressions. In contrast to classical genetic programming, it does not search over tree structures.

Instead, it exploits the LLM’s prior knowledge and iterative prompting to generate new formulas. Each proposal is a Python function body of the fixed form “return ...”, which depends on the variable  $x$  and a parameter array. This design allows the response to be seamlessly integrated with the evaluation program, unifying LLM-based proposal generation with formula evaluation and evolution into a single pipeline. With deliberate guidance and prompts that encourage diverse functional forms, the LLM can also generate expressive formulas. The workflow is illustrated in Figure 3.

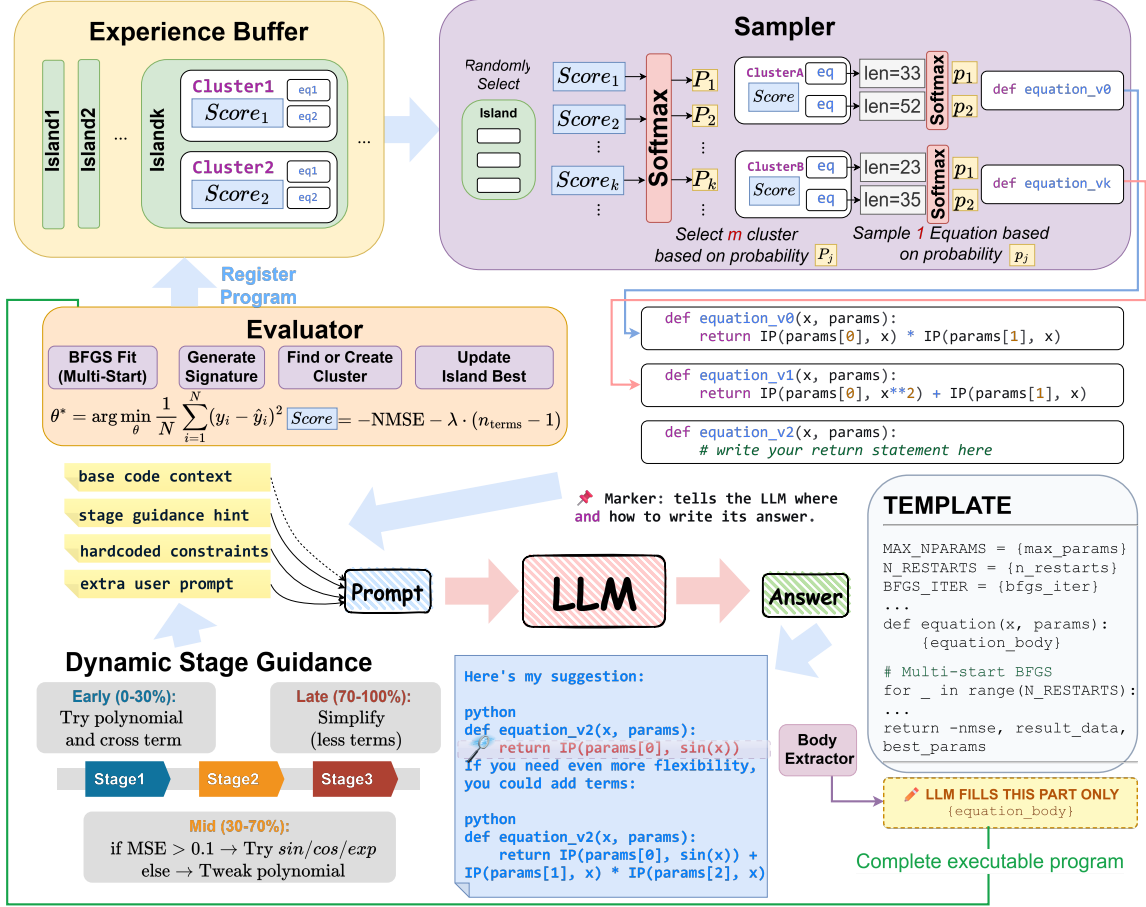


Figure 3: The workflow of LLMSymRegressor. The LLM generates candidate expressions from a prompt constructed by the experience buffer; each candidate is evaluated via BFGS and registered back into the buffer, forming a closed loop.

To guide the search, we maintain an experience buffer that records previously evaluated expressions and their scores. This buffer is organized as a multi-island model, where each island comprises multiple clusters of expressions with identical scores. These cached expressions facilitate prompt construction. The algorithm first randomly selects one of several islands (independent sub-populations) from the experience buffer, then samples clusters via a tempered softmax over their scores, and finally draws one expression from each selected cluster, where shorter expressions are preferred. This multi-level sampling provides the LLM with diverse, high-quality in-context demonstrations. Although performance evaluation and

LLM querying are independent, after observing the existing high-quality expression structures, the LLM can directly propose potentially more suitable expressions. This capability stems from the LLM’s internally accumulated understanding of underlying regularities. In addition, the prompt incorporates stage-wise hints according to the current search progress, encouraging polynomial terms in early stages and trigonometric or exponential functions later, while consistently favoring simplicity. This equips the regressor with the capability to discover complex formulas.

Once a candidate expression is generated, the evaluator assesses its quality by fitting its coefficients via BFGS with multiple restarts. The objective is to minimize the mean squared error on the training data. The score is computed as the negative normalized MSE on the training data to eliminate scale dependence. Then the best expression found during the search is retained, together with its optimized parameters.

The above steps form a closed loop. The algorithm iterates for a fixed number of sampling rounds. In each round, several new expressions are generated and evaluated. Meanwhile, the buffer periodically resets the weaker half of its islands and injects the best individual from the strongest island to maintain diversity. Finally, the algorithm outputs a symbolic expression in homogeneous aggregation form, with coefficients already optimized. The discovered formula can be directly used as a neuron in a task-based neural network, providing an interpretable nonlinear transformation.

### C.3 RLSymRegressor

**RLSymRegressor** is a reinforcement learning agent that discovers a symbolic expression by selecting basis terms from a predefined library. Without considering coefficients, an aggregation function typically takes two common forms:

$$\epsilon_k(\mathbf{x}) = \sum_j x_j^k \quad (k = 1, \dots, K), \quad \psi_r(\mathbf{x}) = \left( \sum_j x_j \right)^r \quad (r = 2, \dots, R), \quad (2)$$

where  $\mathbf{x}$  denotes the input vector,  $K$  denotes the user-specified maximum power, and  $R$  is the order of the interaction, respectively. Accordingly, we adopt these two families as the candidate library. The former provides power terms, and the latter provides interaction terms. Specifically,  $\psi_r$  captures multiplicative interactions among features, which are equivalent to inner-product-like combinations after expansion. At each episode, the agent samples  $m$  terms via a policy network. Duplicates may be retained because of the possible existence of multiple interaction patterns. Then, each term is assigned an independent Ridge coefficient. This reinforcement learning approach does not explicitly specify the coefficients but only selects a structure, letting the parameter optimizer optimize the formula as much as possible. This allows the network to focus on the structure rather than on freely adjustable parameters. The overall workflow is shown in Figure 4.

For a sampled set  $S = \{t_1, \dots, t_m\}$  with  $t_i \in \{\epsilon_k, \psi_r\}$ , the regressor constructs feature vectors by applying each term directly to the input data  $\mathbf{X}$  using fully vectorized tensor operations along the feature dimension. This not only provides a fast aggregation operation but also eliminates explicit loops over individual features  $j$ . The resulting columns are concatenated into a feature matrix  $\mathbf{Z}_{tr} = [t_1(\mathbf{X}_{tr}), \dots, t_m(\mathbf{X}_{tr})]$ , which is then fitted

using Ridge regression on the training set. The  $R^2$  score in the validation set serves as the reward signal, and the policy is optimized via the REINFORCE (Williams, 1992) algorithm to maximize this reward. During training, the network learns a policy regarding how to select terms, aligning the selected structure with the goal of an optimal neuron aggregation function. The policy network is a multilayer perceptron that outputs a probability distribution over candidate terms. The REINFORCE update uses discounted returns with normalization to reduce variance, and the best expression over all episodes is retained.

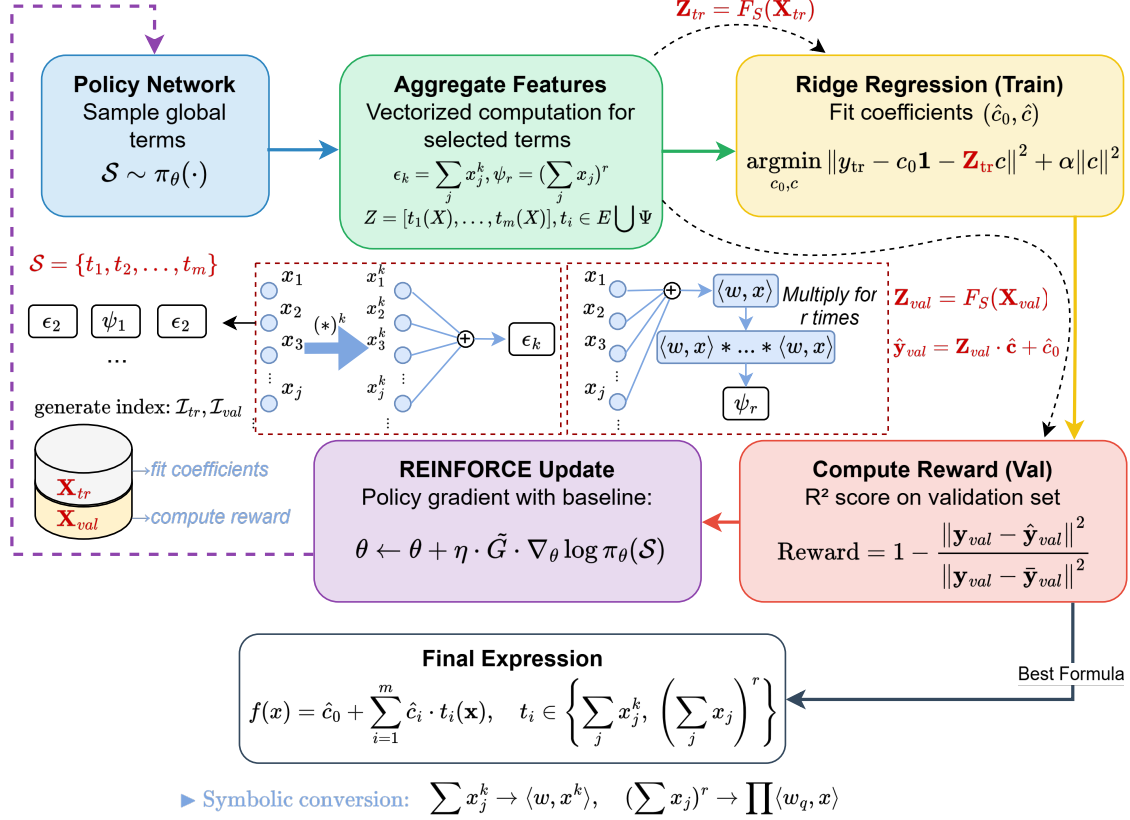


Figure 4: The workflow of RLSymRegressor. The policy network samples basis terms  $(\epsilon_k, \psi_r)$ ; selected terms are vectorized and Ridge-fitted;  $R^2$  rewards in the validation set cause REINFORCE to update; the best expression is returned in inner-product form.

The final output is a symbolic expression of the form:

$$f(\mathbf{x}) = c_0 + \sum_{i=1}^m c_i t_i(\mathbf{x}), \quad (3)$$

where  $c_0$  is the intercept, and  $c_i$  are the Ridge coefficients. It is then converted into an inner-product form, where each  $\epsilon_k$  becomes  $\langle \mathbf{w}, \mathbf{x}^k \rangle$ , and each  $\psi_r$  becomes  $\prod_{q=1}^r \langle \mathbf{w}_q, \mathbf{x} \rangle$ , with the  $\mathbf{w}$ 's serving as symbolic placeholders. The discovered expression is returned as

a string, and can be directly used in neural networks due to its homogeneous aggregation form. The coefficients of the selected expression are later re-optimized by the network’s parameter optimizer.

#### C.4 PolyTensorRegressor

**PolyTensorRegressor** provides a differentiable alternative to symbolic regression for discovering neuronal aggregation functions. By restricting the resultant expression to a tensor-decomposed form, it formulates neuronal discovery as a continuous optimization problem. Given an input matrix  $\mathbf{X} \in \mathbb{R}^{d \times N}$ , the regressor represents a candidate aggregation function through three complementary streams. Specifically, the representation is given by

$$\mathcal{A}_{\text{PolyTensor}}(\mathbf{X}) = \underbrace{\sum_{k=1}^K g_k^{(p)} \left( \mathbf{W}_k^\top \mathbf{X}^{\odot k} \right)}_{\text{polynomial stream}} + \underbrace{\sum_{m=2}^M g_m^{(i)} \sum_{r=1}^R \bigodot_{j=1}^m \left( \mathbf{A}_{r,j}^\top \mathbf{X} \right)}_{\text{interaction stream (CP decomposition)}} + \underbrace{g^{(s)} \mathbf{W}_s^\top \sin(\mathbf{X})}_{\text{periodic component}}, \quad (4)$$

where  $\mathbf{X}^{\odot k}$  denotes the element-wise  $k$ -th power of  $\mathbf{X}$ , and  $\sin(\mathbf{X})$  is applied element-wise. In this expression, the matrices  $\mathbf{W}_k, \mathbf{A}_{r,j}, \mathbf{W}_s \in \mathbb{R}^{d \times d_{\text{out}}}$  are learnable projection matrices, and  $g_k^{(p)}, g_m^{(i)}, g^{(s)} \in [0, 1]$  are differentiable soft gates that determine the extent to which each component is retained. The symbol  $\bigodot_{j=1}^m (\cdot)$  denotes the element-wise (Hadamard) product of the  $m$  matrices produced by the projections. Among the three streams, the polynomial stream captures independent nonlinear transformations of individual features via powers  $\mathbf{X}^{\odot k}$ . The interaction stream models cross-feature dependencies of the order  $m \geq 2$  using a low-rank parameterization based on the CP decomposition (Guo et al., 2011). Instead of explicitly constructing the full tensor and then decomposing it, we directly parameterize the interaction stream, which constrains the tensor rank and enables end-to-end learning of the factor matrices. This reduces the parameter complexity from  $\mathcal{O}(d^m)$  to  $\mathcal{O}(mRd)$  when  $d_{\text{out}}$  is treated as a constant, while preserving the ability to capture high-order feature interactions. The periodic component adds a sinusoidal term to handle cyclic patterns.

The overall workflow is illustrated in Figure 5. To train the regressor, the gates  $g_k^{(p)}, g_m^{(i)}, g^{(s)}$  are optimized jointly with the projection matrices via gradient descent, using a sparsity-inducing regularizer such as an  $\ell_1$  penalty to prune redundant components (Louizos et al., 2017). After convergence, components are retained if their gates exceed a predefined threshold, and are pruned otherwise. This yields a compact formula form:

$$\phi_{\text{PolyTensor}}(\mathbf{x}) = \sum_k \mathbf{W}_k^\top \mathbf{x}^{\odot k} + \sum_m \sum_r \bigodot_{j=1}^m \left( \mathbf{A}_{r,j}^\top \mathbf{x} \right) + \mathbf{W}_s^\top \sin(\mathbf{x}), \quad (5)$$

which can be directly parameterized as a trainable neuron layer. This differentiable search process improves the stability and reproducibility of discovered formulas compared to discrete genetic programming-based symbolic regression, especially for high-dimensional inputs or high-order interactions. The discovered neuron is returned in a form compatible with homogeneous aggregation, and can be embedded into neural network backbones without architectural redesign.

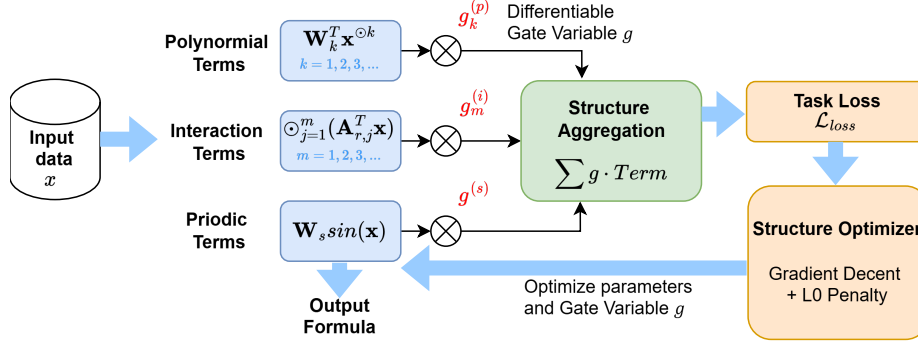


Figure 5: The overall framework of the PolyTensorRegressor, showing the differentiable streams (polynomial, tensor-decomposed interactions, and periodic) and the gating mechanism for structural selection.

## Appendix D. Discovered Neuron Formulas on Benchmark Datasets

| Dataset  | Instances | Features | Task           | Discovered Formula                                                                                                                                                                                                                                                                      |
|----------|-----------|----------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| satimage | 6,430     | 37       | Classification | $-((\mathbf{0.0239} \odot \mathbf{x})^T \mathbf{0.8684}) + (\mathbf{0.0239} \odot \mathbf{x})^T \mathbf{x}$                                                                                                                                                                             |
| phoneme  | 5,404     | 6        | Classification | $0.0394 \sum_i x_i^2 + 0.0243 \sum_i x_i$                                                                                                                                                                                                                                               |
| adult    | 48,842    | 15       | Classification | $-2 \sum_i x_i + \mathbf{x}^T \mathbf{0.1406}$                                                                                                                                                                                                                                          |
| pol      | 15,000    | 49       | Regression     | $-0.1301 \sum_i x_i + 0.0139 - \mathbf{0.0039}^T \mathbf{x}$                                                                                                                                                                                                                            |
| fried    | 40,768    | 11       | Regression     | $-1.7527 \sum_i x_i^3 + 2.7958 \sum_i x_i^2$<br>$-0.1638 \sum_i x_i - 0.4164$                                                                                                                                                                                                           |
| bike     | 17,379    | 12       | Regression     | $(\mathbf{x}^T \mathbf{0.1966})(\mathbf{0.9731}^T \mathbf{x}^{\odot 2}) - (\mathbf{x}^T \mathbf{0.1966})(0.9731 \mathbf{x}^T \mathbf{x})$<br>$-(\mathbf{x}^T \mathbf{0.7017})(\mathbf{0.9731}^T \mathbf{x}^{\odot 2}) + (\mathbf{x}^T \mathbf{0.7017})(0.9731 \mathbf{x}^T \mathbf{x})$ |

Table 2: Formulas discovered by GPSymRegressor on six benchmark datasets. Boldface numerals denote constant vectors;  $\mathbf{x}$  denotes the input vector;  $\odot$  denotes the element-wise product, and  $\mathbf{x}^{\odot k}$  the element-wise  $k$ -th power of  $\mathbf{x}$ .

To evaluate the effectiveness of the task-based neuron framework implemented by TNLearn, we conduct experiments on a collection of datasets sourced from the OpenML repository (Casalicchio et al., 2019). These datasets span a variety of application domains, sample sizes, and feature dimensionalities, providing a fair and comprehensive benchmark. For reproducibility, we use the OpenML ID of each dataset. The datasets used in this work include satimage (ID 182), phoneme (ID 1489), adult (ID 1590), pol (ID 201), fried (ID

564), Bike Sharing Demand (ID 44063), and sarcos (ID 43873). For brevity, we abbreviate Bike Sharing Demand as “bike”.

Given these datasets, the first step is to apply vectorized symbolic regression to discover a formula that captures the underlying pattern of the data. Table 2 lists the formulas automatically obtained by `GPSymRegressor` on six of these datasets. This provides an intuitive example of the formulas that the regressor can discover. For each dataset, we report the number of instances, the number of features, the task type, and the discovered formula. To maintain computational efficiency, we subsample up to 1,000 training instances during the formula searching stage. The numeric constants shown in the table are not fixed. When constructing task-based neurons, they are replaced by learnable parameters, as described in Section 2.2. The resulting formulas then serve as the `neurons` argument when building a task-based network with `MLPRegressor` or `MLPClassifier`.

## Appendix E. Experimental Results with TNLearn

### E.1 Evaluation on Classification and Regression Tasks

We evaluate the effectiveness of the TNLearn library on both regression and classification tasks. All task-based networks are constructed using the `MLPRegressor` or `MLPClassifier` modules with neuron formulas discovered by TNLearn’s regressors.

A comprehensive comparison between task-based neurons and standard linear neurons is conducted with the same network architectures on the datasets. The results are summarized in Table 3. Across the evaluated datasets and architectures, the task-based neuron built by TNLearn consistently surpasses the linear neuron in both regression and classification tasks. This demonstrates that the task-based inductive bias embedded in each neuron via TNLearn significantly enhances feature representation power, eliminating the need to rely on simply stacking a large number of neurons to achieve competitive performance, which is a fundamental limitation of traditional linear networks.

| Dataset  | Architecture | Metric         | Task-based Neuron | Linear Neuron |
|----------|--------------|----------------|-------------------|---------------|
| satimage | 8-3-6        | Accuracy       | <b>0.8896</b>     | 0.8546        |
| phoneme  | 6-2-2        | Accuracy       | <b>0.8529</b>     | 0.8104        |
| adult    | 8-2          | Accuracy       | <b>0.8582</b>     | 0.8546        |
| pol      | 6-1          | R <sup>2</sup> | <b>0.9215</b>     | 0.8926        |
| fried    | 6-2-1        | R <sup>2</sup> | <b>0.9551</b>     | 0.9389        |
| bike     | 8-1          | R <sup>2</sup> | <b>0.8838</b>     | 0.6702        |

Table 3: The performance comparison of linear vs. task-based neurons. The architecture “ $n_1 - n_2 - \dots - n_k$ ” denotes the number of neurons in each layer, with the last layer being the output layer. For classification tasks, the output size equals the number of classes, while for regression, it is 1.

| Dataset  | Arch. (Params) | Score  | Symbolic Expression                                                                                                                                                                                                      |
|----------|----------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| satimage | 5-6 (1560)     | 0.8989 | $(\mathbf{w}_1^T \mathbf{x})(\mathbf{w}_2^T \mathbf{x})(\mathbf{w}_3^T \mathbf{x}) + \mathbf{w}_4^T \mathbf{x}^{\odot 2} + \mathbf{w}_5^T \mathbf{x}^{\odot 3} + (\mathbf{w}_6^T \mathbf{x})(\mathbf{w}_7^T \mathbf{x})$ |
|          | 10-6 (1736)    | 0.8087 | $w_1 \sum_i x_i^3 + w_2 \sum_i x_i - (\mathbf{w}_3 \odot \mathbf{x})^T \mathbf{x} + (\mathbf{w}_4 \odot \mathbf{x})^T (\mathbf{1} \cdot \mathbf{x}^T \mathbf{x})$                                                        |
| phoneme  | 5-2 (235)      | 0.8548 | $(\mathbf{w}_1^T \mathbf{x})(\mathbf{w}_2^T \mathbf{x}) + \mathbf{w}_3^T \mathbf{x}^{\odot 2} + \mathbf{w}_4^T \mathbf{x} + \mathbf{w}_5^T \mathbf{x}$                                                                   |
|          | 10-4-2 (356)   | 0.8363 | $w_1 \sum_i x_i^2 + w_2 \sum_i x_i + \mathbf{w}_3^T \mathbf{x} + \mathbf{x}^T \mathbf{x}$                                                                                                                                |
| adult    | 5-2 (460)      | 0.8610 | $\mathbf{w}_1^T \mathbf{x} + (\mathbf{w}_2^T \mathbf{x})(\mathbf{w}_3^T \mathbf{x}) + \mathbf{w}_4^T \mathbf{x}^{\odot 3} + \mathbf{w}_5^T \mathbf{x}^{\odot 2}$                                                         |
|          | 10-2 (704)     | 0.8561 | $w_1 \sum_i x_i + w_2 - w_3(\mathbf{x}^T \mathbf{w}_4) - (\mathbf{w}_5 \odot \mathbf{x})^T (\mathbf{1} \cdot \mathbf{x}^T \mathbf{x})$                                                                                   |
| bike     | 6-1 (469)      | 0.9079 | $\mathbf{w}_1^T \mathbf{x}^{\odot 3} + \mathbf{w}_2^T \mathbf{x}^{\odot 2} + (\mathbf{w}_3^T \mathbf{x})(\mathbf{w}_4^T \mathbf{x})(\mathbf{w}_5^T \mathbf{x}) + \mathbf{w}_6^T \mathbf{x}$                              |
|          | 8-3-1 (506)    | 0.8709 | $w_1 - ((\mathbf{w}_2^T \mathbf{x})(\mathbf{x}^T \mathbf{x}) + w_3(\mathbf{w}_2^T \mathbf{x})(\mathbf{1}^T \mathbf{x})) - \mathbf{x}^T \mathbf{w}_4$                                                                     |
| fried    | 5-1 (416)      | 0.9586 | $\mathbf{w}_1^T \mathbf{x}^{\odot 3} + \mathbf{w}_2^T \mathbf{x} + (\mathbf{w}_3^T \mathbf{x})(\mathbf{w}_4^T \mathbf{x})(\mathbf{w}_5^T \mathbf{x}) + (\mathbf{w}_6^T \mathbf{x})(\mathbf{w}_7^T \mathbf{x})$           |
|          | 10-4-1 (483)   | 0.1597 | $w_1 \sum_i x_i^4 + w_2 \sum_i x_i^2 - w_3(\mathbf{x}^T \mathbf{x}) + w_4(\mathbf{x}^T \mathbf{x})$                                                                                                                      |
| pol      | 5-1 (1746)     | 0.9908 | $\mathbf{w}_1^T \mathbf{x} + \mathbf{w}_2^T \mathbf{x}^{\odot 2} + (\mathbf{w}_3^T \mathbf{x})(\mathbf{w}_4^T \mathbf{x}) + (\mathbf{w}_5^T \mathbf{x})(\mathbf{w}_6^T \mathbf{x})(\mathbf{w}_7^T \mathbf{x})$           |
|          | 10-1 (1981)    | 0.9774 | $w_1 \sum_i x_i + w_2(\mathbf{x}^T \mathbf{x}) - w_3 \sum_i x_i - (\mathbf{w}_4^T \mathbf{x})(\mathbf{1}^T \mathbf{x})$                                                                                                  |
| sarcos   | 5-1 (581)      | 0.9710 | $\mathbf{w}_1^T \mathbf{x} + \mathbf{w}_2^T \mathbf{x}^{\odot 2} + \mathbf{w}_3^T \mathbf{x} + (\mathbf{w}_4^T \mathbf{x})(\mathbf{w}_5^T \mathbf{x})$                                                                   |
|          | 10-1 (701)     | 0.9693 | $w_1 \sum_i x_i + w_2 + \mathbf{x}^T \mathbf{x} - ((\mathbf{x}^{\odot 2})^T \mathbf{w}_3)(\mathbf{w}_4^T \mathbf{x}^{\odot 3})$                                                                                          |

Table 4: Comparison of task-based neurons and random neurons. For each dataset, the first row uses formulas discovered by **RLSymRegressor**, and the second row uses randomly generated formulas. Classification datasets (satimage, phoneme, adult) use Accuracy; regression datasets (Bike Sharing Demand, fried, pol, sarcos) use  $R^2$ . Architecture “ $n_1$ – $n_2$ – $\dots$ – $n_k$ ” denotes the number of neurons per layer, with the last layer being the output layer, and the number of learnable parameters is shown in parentheses. All numeric constants are replaced by learnable parameters  $w_i$  or  $\mathbf{w}_i$  (vectors bold).

Furthermore, to verify if the formulas discovered by symbolic regression truly capture the underlying data patterns rather than arbitrary mathematical coincidences, we compare networks built with discovered formulas using **RLSymRegressor** against networks built with randomly generated formulas. The random formulas are generated by a tree-based generator using the same operators as **GPSymRegressor**, then subjected to multiple mutations and crossovers without fitness-based parent selection, and finally simplified with **SymPy**. For a fair comparison, they are generated with a comparable complexity to the discovered formulas in terms of the number of terms and occurrences of  $\mathbf{x}$ . As shown in Table 4, the task-based network constructed by **TNLearn** consistently outperforms the random network across nearly all datasets, achieving higher regression  $R^2$  or higher classification accuracy, even with

smaller architectures. This advantage is achieved with fewer parameters, showing that task-based neurons can represent features effectively without increasing model capacity. These results also validate the symbolic regression engine in **TNLearn** as an effective extractor of task-relevant inductive biases, whereas random formulas lack such targeted representational effectiveness.

Despite the remarkable success of quadratic neurons in many tasks, **TNLearn** still achieves superior performance. We compare the proposed task-based neurons with four representative quadratic neuron variants (Bu and Karpadne, 2021; Fan et al., 2018; Goyal et al., 2020; Xu et al., 2022) on the benchmark datasets. We construct all these networks using **TNLearn** by specifying the corresponding quadratic neuron expressions. This provides a convenient way to benchmark different neuron designs. The task-based neurons compared here are those listed in Table 2.

As shown in Table 5, task-based neurons achieve superior or competitive performance. Unlike hand-crafted quadratic neurons with fixed parametric forms, **GPSymRegressor** automatically adapts the neuronal form to the characteristics of each dataset, providing greater flexibility. This flexibility is reflected in the diverse formulas discovered across datasets (Table 2). Overall, these results demonstrate that task-based neurons outperform classical quadratic neurons, which also establishes **TNLearn** as a convenient platform for benchmarking different neuron designs.

| Dataset  | Architecture | Ours   | Bu and Karpadne (2021) | Fan et al. (2018) | Goyal et al. (2020) | Xu et al. (2022) |
|----------|--------------|--------|------------------------|-------------------|---------------------|------------------|
| pol      | 12-5-1       | 0.9950 | 0.9762                 | 0.9874            | 0.8678              | 0.9929           |
| fried    | 4-1          | 0.9562 | 0.1586                 | 0.1561            | 0.1589              | 0.9556           |
| bike     | 8-3-1        | 0.9250 | 0.8910                 | 0.9028            | 0.3880              | 0.9138           |
| satimage | 4-3-6        | 0.8756 | 0.7939                 | 0.7426            | 0.6726              | 0.7729           |
| phoneme  | 5-3-2        | 0.8548 | 0.7669                 | 0.7928            | 0.7872              | 0.8131           |

Table 5: Comparison of task-based neurons and four quadratic neuron methods (Bu and Karpadne, 2021; Fan et al., 2018; Goyal et al., 2020; Xu et al., 2022). Classification datasets (satimage, phoneme) use Accuracy; regression datasets use  $R^2$ . Architecture “ $n_1-n_2-\dots-n_k$ ” denotes the number of neurons in each layer, with the last layer being the output layer.

## E.2 Evaluation on Images

To validate the effectiveness of task-based neurons for convolutional networks in a controlled setting, we conduct experiments on the MNIST handwritten digit recognition dataset (LeCun et al., 1998), which contains 60,000 training and 10,000 test grayscale images of size  $28 \times 28$  across 10 classes. We construct a lightweight convolutional backbone consisting of convolutional layers, followed by global average pooling and a fully connected classifier. We compare two variants: the standard **Conv2d** in the PyTorch modules, which uses the conventional linear convolution  $y = \mathbf{W} * \mathbf{x} + b$ , and **TNConv2d** in the **TNLearn** modules, which replaces the standard convolution with a task-driven neuronal convolution. As a data type more complex than tabular data, images require more tricks for neuron discovery because

their input dimensionality does not match that of the regressors. The aggregation function of this neuron is obtained through a two-stage discovery process. First, an autoencoder is used to reduce the dimensionality of image patches. Then, vectorized symbolic regression is applied to the resulting low-dimensional data. The autoencoder compresses the 2D spatially structured image into a compact low-dimensional latent vector while preserving essential task-relevant semantic information. It serves not only as a necessary bridge between image space and symbolic regression, but also provides dimensionality reduction, which reduces the computational cost of neuron discovery and improves convergence.

Here, we discover a compact mathematical expression via `LLMSymRegressor`, which is subsequently implemented as the neuron formula. Both models share identical architecture, optimizer (Adam, learning rate  $10^{-3}$ ), batch size (64), and training protocol (5 epochs). Each experiment is repeated 5 times, and the average results are reported.

Table 6: Comparison of Standard Conv2d and TNConv2d on MNIST.

| Epoch | Standard Conv2d |          | TNConv2d (TNLearn) |              |
|-------|-----------------|----------|--------------------|--------------|
|       | Loss            | Acc. (%) | Loss               | Acc. (%)     |
| 1     | 0.1614          | 98.49    | 0.1826             | <b>98.74</b> |
| 2     | 0.0525          | 98.68    | 0.0568             | <b>99.02</b> |
| 3     | 0.0394          | 99.05    | 0.0402             | 99.05        |
| 4     | 0.0293          | 99.04    | 0.0308             | <b>99.18</b> |
| 5     | 0.0252          | 99.05    | 0.0257             | <b>99.21</b> |

Table 6 summarizes the training loss and test accuracy across all epochs. `TNConv2d` consistently achieves competitive performance while maintaining a comparable final training loss. In particular, `TNConv2d` exhibits faster early convergence.

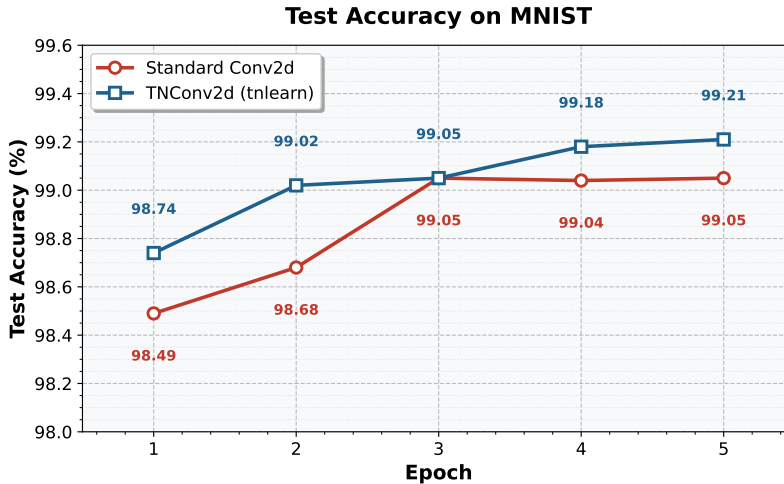


Figure 6: The test accuracy curves for the standard Conv2d vs. `TNConv2d` on the MNIST dataset over 5 epochs.

The corresponding test accuracy curves are visualized in Figure 6. Task-specific symbolic neurons are able to capture more intricate pixel interactions than a purely linear kernel. The trend of the training loss shows that `TNConv2d` achieves a convergence speed comparable to that of the standard `Conv2d`, indicating that the increased complexity of the task-driven neuron does not compromise optimization efficiency. This indicates that the task-specific neuron design strikes a favorable balance between representational power and trainability.

## References

- Deaglan J Bartlett, Harry Desmond, and Pedro G Ferreira. Exhaustive symbolic regression. *IEEE Transactions on Evolutionary Computation*, 28(4):950–964, 2023. doi:[10.1109/tevc.2023.3280250](https://doi.org/10.1109/tevc.2023.3280250).
- Jie Bu and Anuj Karpatne. Quadratic residual networks: A new class of neural networks for solving forward and inverse problems in physics involving pdes. In *Proceedings of the 2021 SIAM International Conference on Data Mining (SDM)*, pages 675–683. SIAM, 2021. doi:[10.1137/1.9781611976700.76](https://doi.org/10.1137/1.9781611976700.76).
- Giuseppe Casalicchio, Jakob Bossek, Michel Lang, Dominik Kirchhoff, Pascal Kerschke, Benjamin Hofner, Heidi Seibold, Joaquin Vanschoren, and Bernd Bischl. Openml: An r package to connect to the machine learning platform openml. *Computational Statistics*, 34(3):977–991, 2019. doi:[10.1007/s00180-017-0742-2](https://doi.org/10.1007/s00180-017-0742-2).
- Grigorios G Chrysos, Stylianos Moschoglou, Giorgos Bouritsas, Jiankang Deng, Yannis Panagakis, and Stefanos Zafeiriou. Deep polynomial neural networks. *IEEE transactions on pattern analysis and machine intelligence*, 44(8):4021–4034, 2021. doi:[10.1109/tpami.2021.3058891](https://doi.org/10.1109/tpami.2021.3058891).
- Grigorios G Chrysos, Markos Georgopoulos, Jiankang Deng, Jean Kossaifi, Yannis Panagakis, and Anima Anandkumar. Augmenting deep classifiers with polynomial neural networks. In *European Conference on Computer Vision*, pages 692–716. Springer, 2022. doi:[10.1007/978-3-031-19806-9\\_40](https://doi.org/10.1007/978-3-031-19806-9_40).
- Nichael Lynn Cramer. A representation for the adaptive generation of simple sequential programs. In *Proceedings of the First International Conference on Genetic Algorithms and Their Applications*, pages 183–187. Lawrence Erlbaum Associates, 1985. URL <https://dl.acm.org/doi/10.5555/645511.657085>.
- Feng-Lei Fan, Dayang Wang, Hengtao Guo, Qikui Zhu, Pingkun Yan, Ge Wang, and Hengyong Yu. On a sparse shortcut topology of artificial neural networks. *IEEE Transactions on Artificial Intelligence*, 3(4):595–608, 2021. doi:[10.1109/tai.2021.3128132](https://doi.org/10.1109/tai.2021.3128132).
- Feng-Lei Fan, Hang-Cheng Dong, Zhongming Wu, Lecheng Ruan, Tiejong Zeng, Yiming Cui, and Jing-Xiao Liao. One neuron saved is one neuron earned: On parametric efficiency of quadratic networks. *IEEE Transactions on Pattern Analysis & Machine Intelligence*, 47(11):9702–9717, 2025a. doi:[10.1109/tpami.2025.3588894](https://doi.org/10.1109/tpami.2025.3588894).

- Feng-Lei Fan, Yingxin Li, Tiejong Zeng, Fei Wang, and Hanchuan Peng. Towards neuroai: introducing neuronal diversity into artificial neural networks. *Med-X*, 3(1):2, 2025b. doi:[10.1007/s44258-024-00042-2](https://doi.org/10.1007/s44258-024-00042-2).
- Feng-Lei Fan, Meng Wang, Hang-Cheng Dong, Jianwei Ma, and Tiejong Zeng. No one-size-fits-all neurons: Task-based neurons for artificial neural networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026. doi:[10.1109/TPAMI.2026.3704202](https://doi.org/10.1109/TPAMI.2026.3704202).
- Fenglei Fan, Wenxiang Cong, and Ge Wang. A new type of neurons for machine learning. *International journal for numerical methods in biomedical engineering*, 34(2):e2920, 2018. doi:[10.1002/cnm.2920](https://doi.org/10.1002/cnm.2920).
- Fenglei Fan, Hongming Shan, Mannudeep K Kalra, Ramandeep Singh, Guhan Qian, Matthew Getzin, Yueyang Teng, Juergen Hahn, and Ge Wang. Quadratic autoencoder (q-ae) for low-dose ct denoising. *IEEE transactions on medical imaging*, 39(6):2035–2050, 2019. doi:[10.1109/tmi.2019.2963248](https://doi.org/10.1109/tmi.2019.2963248).
- Mohit Goyal, Rajan Goyal, and Brejesh Lall. Improved polynomial neural networks with normalised activations. In *2020 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2020. doi:[10.1109/ijcnn48605.2020.9207535](https://doi.org/10.1109/ijcnn48605.2020.9207535).
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023. doi:[10.48550/arXiv.2312.00752](https://doi.org/10.48550/arXiv.2312.00752).
- Weiwei Guo, Irene Kotsia, and Ioannis Patras. Tensor learning for regression. *IEEE Transactions on Image Processing*, 21(2):816–827, 2011. doi:[10.1109/TIP.2011.2165291](https://doi.org/10.1109/TIP.2011.2165291).
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. doi:[10.1109/cvpr.2016.90](https://doi.org/10.1109/cvpr.2016.90).
- Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017. doi:[10.1109/cvpr.2017.243](https://doi.org/10.1109/cvpr.2017.243).
- Yiyang Jiang, Fan Yang, Hengliang Zhu, Dian Zhou, and Xuan Zeng. Nonlinear cnn: improving cnns with quadratic convolutions. *Neural Computing and Applications*, 32(12):8507–8516, 2020. doi:[10.1007/s00521-019-04316-4](https://doi.org/10.1007/s00521-019-04316-4).
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi:[10.1109/5.726791](https://doi.org/10.1109/5.726791).
- Sai Li, Peng Kou, Miao Ma, Haoyu Yang, Shuo Huang, and Zhengyi Yang. Application of semi-supervised learning in image classification: Research on fusion of labeled and unlabeled data. *IEEE Access*, 12:27331–27343, 2024. doi:[10.1109/access.2024.3367772](https://doi.org/10.1109/access.2024.3367772).
- Jing-Xiao Liao, Hang-Cheng Dong, Zhi-Qi Sun, Jinwei Sun, Shiping Zhang, and Feng-Lei Fan. Attention-embedded quadratic network (qttnet) for effective and interpretable

- bearing fault diagnosis. *IEEE Transactions on Instrumentation and Measurement*, 72: 1–13, 2023. doi:[10.1109/tim.2023.3259031](https://doi.org/10.1109/tim.2023.3259031).
- Jing-Xiao Liao, Chao He, Jipu Li, Jinwei Sun, Shiping Zhang, and Xiaoge Zhang. Classifier-guided neural blind deconvolution: A physics-informed denoising module for bearing fault diagnosis under noisy conditions. *Mechanical Systems and Signal Processing*, 222:111750, 2025. doi:[10.1016/j.ymssp.2024.111750](https://doi.org/10.1016/j.ymssp.2024.111750).
- Christos Louizos, Max Welling, and Diederik P Kingma. Learning sparse neural networks through  $l_0$  regularization. *arXiv preprint arXiv:1712.01312*, 2017. doi:[10.48550/arXiv.1712.01312](https://doi.org/10.48550/arXiv.1712.01312).
- Pranav Mantini and Shishr K Shah. Cqnn: Convolutional quadratic neural networks. In *2020 25th International Conference on Pattern Recognition (ICPR)*, pages 9819–9826. IEEE, 2021. doi:[10.1109/icpr48806.2021.9413207](https://doi.org/10.1109/icpr48806.2021.9413207).
- Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, pages 807–814, 2010. URL <https://icml.cc/Conferences/2010/papers/432.pdf>.
- Tuan Nguyen, Alireza Kashani, Tuan Ngo, and Stphane Bordas. Deep neural network with high-order neuron for the prediction of foamed concrete strength. *Computer-Aided Civil and Infrastructure Engineering*, 34(4):316–332, 2019. doi:[10.1111/mice.12422](https://doi.org/10.1111/mice.12422).
- Hanchuan Peng, Peng Xie, Lijuan Liu, Xiuli Kuang, Yimin Wang, Lei Qu, Hui Gong, Shengdian Jiang, Anan Li, Zongcai Ruan, et al. Morphological diversity of single neurons in molecularly defined cell types. *Nature*, 598(7879):174–181, 2021. doi:[10.1038/s41586-021-03941-1](https://doi.org/10.1038/s41586-021-03941-1).
- Michael Schmidt and Hod Lipson. Distilling free-form natural laws from experimental data. *science*, 324(5923):81–85, 2009. doi:[10.1126/science.1165893](https://doi.org/10.1126/science.1165893).
- Rupesh Kumar Srivastava, Klaus Greff, and Jrgen Schmidhuber. Highway networks. *arXiv preprint arXiv:1505.00387*, 2015. URL <https://arxiv.org/abs/1505.00387>.
- Yuheng Tang, Chaoyong Zhang, Jianzhao Wu, Yang Xie, Weiming Shen, and Jun Wu. Deep learning-based bearing fault diagnosis using a trusted multiscale quadratic attention-embedded convolutional neural network. *IEEE transactions on instrumentation and measurement*, 73:1–15, 2024. doi:[10.1109/tim.2024.3374311](https://doi.org/10.1109/tim.2024.3374311).
- Runxiang Wang, Boxiao Wang, Kai Li, Yifan Zhang, and Jian Cheng. Drsrr: Llm based scientific equation discovery with dual reasoning from data and experience. *arXiv preprint arXiv:2506.04282*, 2025. doi:[10.48550/arXiv.2506.04282](https://doi.org/10.48550/arXiv.2506.04282).
- Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992. doi:[10.1007/BF00992696](https://doi.org/10.1007/BF00992696).
- Zhihong Wu, Lishuang Wang, Kebin Sun, Zhuozhao Li, and Ran Cheng. Enabling population-level parallelism in tree-based genetic programming for gpu acceleration. *IEEE Transactions on Evolutionary Computation*, 2026. doi:[10.1109/tevc.2026.3663396](https://doi.org/10.1109/tevc.2026.3663396).

Chenhui Xu, Fuxun Yu, Jinjun Xiong, and Xiang Chen. Quadranet v2: Efficient and sustainable training of high-order neural networks with quadratic adaptation. In *2026 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1365–1373. IEEE, 2026. doi:[10.1109/WACV61042.2026.00139](https://doi.org/10.1109/WACV61042.2026.00139).

Zirui Xu, Fuxun Yu, Jinjun Xiong, and Xiang Chen. Quadralib: A performant quadratic neural network library for architecture optimization and design exploration. *Proceedings of Machine Learning and Systems*, 4:503–514, 2022. doi:[10.48550/arXiv.2204.01701](https://doi.org/10.48550/arXiv.2204.01701).